CODE ANALYSIST REPORT

SEPTEMBER 2026



FUNCTIONAL PROGRAMMING

MODULE 2 - Effect-free Programming and Declarative Paradigm

COMPILED BY:

Mohamad Akbar Noviandi

202410370110388

Pemrograman Fungsional G

1. Pure Function

15 Point

Exercise 1: functional analysis

Complete the code by calling add_bonus2() and run the program multiple times (2–3 times) and observe the changes in both result and exam_scores. Why does add_bonus2() modify the original exam_scores data even though it does not use the global keyword? Explain your analysis. Based on its characteristics, does the add_bonus2() function qualify as a Pure Function or an Impure Function? Explain your reasons.

Screenshot of the answer (must include student ID / NIM)

```
# 202410370110388
#data awal
exam_scores = [70, 80, 85]
print("Original exam_scores:", exam_scores)
```

✓ 0.0s Python

```
Original exam_scores: [70, 80, 85]
```

```
def add_bonus2(bonus):
    for i in range(len(exam_scores)):
        exam_scores[i] = exam_scores[i] + bonus
    return exam_scores

# 202410370110388
print("Before:", exam_scores)
result = add_bonus2(5)
print("New result:", result)
print("Original exam_scores be:", exam_scores)
```

✓ 0.0s Python

```
Before: [70, 80, 85]
New result: [75, 85, 90]
Original exam_scores be: [75, 85, 90]
```

Explanations:

Fungsi `add_bonus2()` mengubah data asli `exam_scores` karena dalam Python, ketika kita memodifikasi objek yang bersifat mutable (seperti list), perubahan tersebut akan mempengaruhi objek asli yang direferensikan. Dalam hal ini, `exam_scores` adalah sebuah list, dan ketika kita menambahkan bonus ke setiap elemen dalam list tersebut, kita sebenarnya memodifikasi isi dari list itu sendiri, bukan membuat salinan baru.

2. Immutability

15 Point

Exercise 2: Data duplication with shallow and deep copy

```
user = { "name" : "putri", "scores" : [90, 85]}
```

1. Create duplicates of the user variable using both Shallow Copy (store it in a variable named user_shallow) and Deep Copy (store it in a variable named user_deep).
2. In each copied object (user_shallow and user_deep), change the scores value from 90 to 75.
3. Check the memory addresses of the modified index, and compare them with the memory address in the original user variable.

Observe the original data output and its memory address, then explain why the shallow copy and deep copy results are different!

Screenshot of the answer (must include student ID / NIM)

```
# 202410370110388
import copy

user = { "name": "putri", "scores" : [90, 85]}
```

✓ 0.0s Python

```
# 202410370110388
# 1. Buat duplikat
user_shallow = copy.copy(user)
user_deep = copy.deepcopy(user)

# Kondisi awal sebelum modifikasi
print("Data awal  :", user)
print("id(user['scores'])         :", id(user["scores"]))
print("id(user_shallow['scores']) :", id(user_shallow["scores"]))
print("id(user_deep['scores'])    :", id(user_deep["scores"]))
print("id(user['scores'][0])      :", id(user["scores"][0]), "(nilai 90 asli)")
```

✓ 0.0s Python

```
Data awal  : {'name': 'putri', 'scores': [90, 85]}
id(user['scores'])         : 1368477903616
id(user_shallow['scores']) : 1368477903616
id(user_deep['scores'])    : 1368477902144
id(user['scores'][0])      : 140717146333080 (nilai 90 asli)
```

```python
# 202410370110388
# 2. Ubah scores 90 -> 75 pada Deep Copy dulu
user_deep["scores"][0] = 75

print("=== Setelah Deep Copy dimodifikasi ===")
print("user      :", user)
print("user_deep :", user_deep)
print("id(user['scores'][0])      :", id(user["scores"][0]))
print("id(user_deep['scores'][0]) :", id(user_deep["scores"][0]))
print("Alamat sama?               :", id(user["scores"][0]) == id(user_deep
["scores"][0]))
```

✓ 0.0s Python

```
=== Setelah Deep Copy dimodifikasi ===
user      : {'name': 'putri', 'scores': [90, 85]}
user_deep : {'name': 'putri', 'scores': [75, 85]}
id(user['scores'][0])      : 140717146333080
id(user_deep['scores'][0]) : 140717146332600
Alamat sama?               : False
```

```python
# 202410370110388
# 2. Ubah scores 90 -> 75 pada Shallow Copy
user_shallow["scores"][0] = 75

print("=== Setelah Shallow Copy dimodifikasi ===")
print("user         :", user)
print("user_shallow :", user_shallow)
print("id(user['scores'][0])         :", id(user["scores"][0]))
print("id(user_shallow['scores'][0]) :", id(user_shallow["scores"][0]))
print("Alamat sama?                  :", id(user["scores"][0]) == id
(user_shallow["scores"][0]))
print("List scores sama objek?       :", user["scores"] is user_shallow
["scores"])
```

✓ 0.0s Python

```
=== Setelah Shallow Copy dimodifikasi ===
user         : {'name': 'putri', 'scores': [75, 85]}
user_shallow : {'name': 'putri', 'scores': [75, 85]}
id(user['scores'][0])         : 140717146332600
id(user_shallow['scores'][0]) : 140717146332600
Alamat sama?                  : True
List scores sama objek?       : True
```

Explanations:

Hasil Shallow Copy dan Deep Copy berbeda karena cara mereka menangani objek yang bersifat mutable. Shallow Copy hanya membuat salinan dari referensi objek, sehingga jika kita mengubah nilai pada objek yang bersifat mutable (seperti list atau dictionary) dalam salinan, perubahan tersebut akan mempengaruhi objek asli. Sebaliknya, Deep Copy membuat salinan lengkap dari objek beserta semua objek yang direferensikan di dalamnya, sehingga perubahan pada salinan tidak akan mempengaruhi objek asli. Oleh karena itu, ketika kita mengubah nilai scores pada user_shallow, perubahan tersebut juga terlihat pada user asli, sedangkan perubahan

pada user_deep tidak mempengaruhi user asli.

3. Iterator

20 Point

Exercise 3: Using iter() and next() to Process Tuple Data

Create a tuple named sensor_data = (23.5, 24.0, 22.8, 21.1). Convert this tuple into an iterator object using the iter() function. Retrieve and print the first two values from the iterator using the next() function, and then print the remaining data that hasn't been fetched yet using a standard for loop!

1. Observe the output of the for loop. Why does the for loop immediately print the remaining data (the third data point) instead of iterating from the first element?
2. Explain how the for loop mechanism works behind the scenes in Python in handling iterator objects without using the next() function explicitly?

Screenshot of the answer (must include student ID / NIM)

```python
# 202410370110388
sensor_data = (23.5, 24.0, 22.8, 21.1)

# Ubah tuple menjadi iterator
sensor_iterator = iter(sensor_data)
print("Tipe objek:", type(sensor_iterator))

# Ambil dua nilai pertama dengan next()
print("Data ke-1:", next(sensor_iterator))
print("Data ke-2:", next(sensor_iterator))

# Tampilkan data yang tersisa dengan for loop biasa
print("Data yang tersisa:")
for value in sensor_iterator:
    print(value)
```

✓ 0.0s Python

```
Tipe objek: <class 'tuple_iterator'>
Data ke-1: 23.5
Data ke-2: 24.0
Data yang tersisa:
22.8
21.1
```

```python
# 202410370110388
# Ekuivalen for loop secara manual (untuk membuktikan mekanismenya)
sensor_iterator2 = iter(sensor_data)
next(sensor_iterator2)
next(sensor_iterator2)
(variable) sensor_iterator2: Iterator[float]
it = iter(sensor_iterator2)
print("iter(iterator) adalah objek yang sama?", it is sensor_iterator2)

while True:
    try:
        value = next(it)
    except StopIteration:
        break
    print(value)
```

✓ 0.0s Python

```
iter(iterator) adalah objek yang sama? True
22.8
21.1
```

Explanations:

1. For loop langsung menampilkan data yang tersisa (data ketiga) karena iterator telah mengambil dua nilai pertama menggunakan fungsi next(). Setelah dua nilai pertama diambil, posisi iterator berada pada elemen ketiga, sehingga for loop mulai dari posisi tersebut dan menampilkan data yang tersisa.
2. Mekanisme for loop bekerja dengan memanggil fungsi iter() pada objek yang diberikan, yang menghasilkan sebuah iterator. For loop kemudian menggunakan fungsi next() secara internal untuk mengambil elemen satu per satu dari iterator hingga mencapai akhir. Ketika for loop dijalankan, ia secara otomatis menangani pemanggilan next() dan menghentikan iterasi ketika StopIteration exception terjadi, sehingga kita tidak perlu memanggil next() secara eksplisit.

4. List Comprehension

15 Point

Exercise 4: Convert to List Comprehension

Convert the following code into a list comprehension:

```
result = []
for letter in "universitas":
    if letter not in "aiueo":
        result.append(letter.upper())

print(result)
```

```
['N', 'V', 'R', 'S', 'T', 'S']
```

Screenshot of the answer (must include student ID / NIM)





Explanations of Code:

List Comprehension adalah cara yang lebih ringkas dan efisien untuk membuat list baru dengan menggunakan sintaks yang lebih sederhana. Dengan List

Comprehension, kita dapat mengekspresikan operasi pembuatan list dalam satu baris kode, yang membuatnya lebih mudah dibaca dan dipahami dibandingkan dengan menggunakan loop tradisional.

5. Generator Expression

15 Point

Exercise 5: Create Generator Expression

Create a Generator Expression that produces numbers from 1 to 5, then display its values using next() inside a while loop. Add exception handling to manage the scenario when the data runs out (StopIteration), and print a message indicating that the iterator has finished.

Screenshot of the answer (must include student ID / NIM)

```python
# 202410370110388
numbers_gen = (x for x in range(1, 6))
print("Tipe objek:", type(numbers_gen))

while True:
    try:
        value = next(numbers_gen)
        print("Nilai:", value)
    except StopIteration:
        print("Iterator telah selesai: tidak ada data yang tersisa.")
        break
```

✓ 0.0s Python

```
Tipe objek: <class 'generator'>
Nilai: 1
Nilai: 2
Nilai: 3
Nilai: 4
Nilai: 5
Iterator telah selesai: tidak ada data yang tersisa.
```

Explanations of Code:
Generator Expression adalah cara untuk membuat generator secara ringkas menggunakan sintaks yang mirip dengan List Comprehension, tetapi menggunakan tanda kurung biasa ( ) alih-alih tanda kurung siku [ ]. Generator Expression menghasilkan objek generator yang dapat menghasilkan nilai satu per satu saat diminta, sehingga lebih efisien dalam penggunaan memori dibandingkan dengan membuat list secara langsung.

6. Generator & Lazy Evaluation

20 Point
Exercise 6: Create Generator Instructions: 1. Create a generator function named filter_emergency_log(log_list) that accepts the log data. 2. The function must evaluate each log line. If a log line starts with the keyword "ERROR", emit (yield) that log line progressively. 3. In the main program, invoke the generator function and store it in a variable named error_bucket. 4. Manually retrieve the first and second values using the next() function. If the server log data volume increases to 10,000,000 rows, why is using a generator function with yield much safer for computer memory compared to a regular function that collects all the results into a list and returns them all at once with return?
Screenshot of the answer (must include student ID / NIM)

```python
server_logs = [
"INFO: User login successful",
"ERROR: Database connection failed",
"INFO: Page rendered in 25ms",
"ERROR: Unauthorized access attempt",
"INFO: User logout successful"
]
```

0.0s Python

```python
def filter_emergency_log(log_list):
    """
    pre : log_list adalah sequence/iterable berisi string log
    post: menghasilkan (yield) satu per satu baris log yang diawali "ERROR"
    """
    for log in log_list:
        if log.startswith("ERROR"):
            yield log
```

0.0s Python

```python
# 202410370110388
# Program utama
error_bucket = filter_emergency_log(server_logs)
print("Tipe objek:", type(error_bucket))   # generator, belum ada log yang diproses

# Ambil nilai pertama dan kedua secara manual
print("Log darurat ke-1:", next(error_bucket))
print("Log darurat ke-2:", next(error_bucket))
```

0.0s Python

```
Tipe objek: <class 'generator'>
Log darurat ke-1: ERROR: Database connection failed
Log darurat ke-2: ERROR: Unauthorized access attempt
```

Explanations:

Penggunaan Generator Function dengan yield jauh lebih aman untuk memori komputer dibandingkan dengan fungsi biasa yang mengumpulkan seluruh hasil ke dalam list dan mengembalikannya sekaligus menggunakan return karena Generator Function menghasilkan nilai satu per satu secara bertahap, sehingga hanya menyimpan satu nilai dalam memori pada satu waktu. Hal ini mengurangi penggunaan memori secara signifikan, terutama ketika berhadapan dengan volume data yang sangat besar, seperti 10.000.000 baris log. Sebaliknya, fungsi biasa yang mengumpulkan seluruh hasil ke dalam list akan memerlukan memori yang besar untuk menyimpan semua nilai sekaligus, yang dapat menyebabkan penggunaan memori yang berlebihan dan bahkan potensi crash pada sistem jika memori tidak cukup. Dengan menggunakan yield, kita dapat memproses data secara efisien dan menghindari masalah memori yang terkait dengan pengumpulan data besar sekaligus.

CODE ANALYSIS ASSESSMENT

Code Analysis	Points (Total 20%)
Exercise Implementation	**10**
Seluruh Exercise selesai dikerjakan dengan benar (poin akumulasi dari seluruh exercise yang ada).	100
modifikasi kode tidak berhasil diselesaikan dengan benar	-50% (setiap exercise)
Exercise dikerjakan, namun kode/output salah	-30% (setiap exercise)
Jawaban tidak mengandung identitas praktikan (Nama/NIM)	-20% (setiap exercise)
Selesai melebihi jam praktikum	-100% (setiap exercise)
Practicum Report	**10**
Menggunakan template resmi, seluruh bagian explanation diisi dengan jawaban yang benar dan jelas	100
Jawaban salah atau jawaban kurang tepat	-50% (setiap exercise)
Penjelasan jawaban ditemukan identik dengan praktikan lain	-100% (setiap exercise)
Report dikirim melebihi dateline (saat praktikum)	-40 point (dari nilai akhir)